# HW1 - Loop Unrolling

## ➢ Introduction

Loop unrolling is a common compiler optimization technique aimed at improving execution efficiency by reducing loop overhead and increasing instruction-level parallelism. This report investigates the effect of loop unrolling on execution time and speedup for different input sizes, focusing on unrolling degrees of 4, 8, 16, and 64.

## ➢ Procedure and Results

- **Experimental Setup**

- The experiment was conducted using the native Linux, using a simulator called VMware Workstation.
- The source code implements normal loops and unrolled loops with degrees 4, 8, 16, and 64.
- The array sizes tested were: 1024, 1024*1024, and 16*1024*1024.
- For each configuration, the code was executed 17 times, and the outliers (maximum and minimum values) were excluded from the average.

| 1024 | | | | | |
|---|---|---|---|---|---|
| Trial | Normal (s) | Degree-4 (s) | Degree-8 (s) | Degree-16 (s) | Degree-64 (s) |
| 1 | 0.00000252 | 0.00000105 | 0.00000097 | 0.00000101 | 0.00000106 |
| 2 | 0.00000215 | 0.00000113 | 0.00000109 | 0.00000104 | 0.00000114 |
| 3 | 0.00000225 | 0.0000011 | 0.00000107 | 0.00000102 | 0.00000114 |
| 4 | 0.00000214 | 0.0000012 | 0.00000118 | 0.00000103 | 0.00000176 |
| 5 | 0.00000248 | 0.0000017 | 0.00000104 | 0.00000125 | 0.00000111 |
| 6 | 0.00000217 | 0.00000127 | 0.00000108 | 0.00000102 | 0.00000108 |
| 7 | 0.00000206 | 0.00000106 | 0.00000116 | 0.00000106 | 0.00000111 |
| 8 | 0.00000207 | 0.00000106 | 0.00000104 | 0.0000011 | 0.0000015 |
| 9 | 0.00000215 | 0.00000106 | 0.00000117 | 0.00000104 | 0.00000108 |
| 10 | 0.00000215 | 0.00000119 | 0.00000105 | 0.00000109 | 0.00000114 |
| 11 | 0.00000214 | 0.00000106 | 0.00000111 | 0.00000113 | 0.00000109 |
| 12 | 0.00000214 | 0.00000113 | 0.0000011 | 0.00000108 | 0.00000108 |
| 13 | 0.00000208 | 0.00000114 | 0.00000104 | 0.00000109 | 0.00000115 |
| 14 | 0.00000206 | 0.00000113 | 0.00000109 | 0.0000011 | 0.00000119 |
| 15 | 0.0000022 | 0.00000114 | 0.00000104 | 0.00000109 | 0.00000114 |
| 16 | 0.00000213 | 0.00000106 | 0.00000116 | 0.00000104 | 0.00000114 |
| 17 | 0.00000234 | 0.00000188 | 0.00000193 | 0.00000104 | 0.00000119 |

| 1024*1024 | | | | | |
|---|---|---|---|---|---|
| Trial | Normal (s) | Degree-4 (s) | Degree-8 (s) | Degree-16 (s) | Degree-64 (s) |
| 1 | 0.002171 | 0.001221 | 0.001142 | 0.00113 | 0.001603 |
| 2 | 0.002361 | 0.001249 | 0.001187 | 0.001815 | 0.001247 |
| 3 | 0.002244 | 0.001135 | 0.001118 | 0.001193 | 0.00126 |
| 4 | 0.002169 | 0.001159 | 0.001432 | 0.001407 | 0.001646 |
| 5 | 0.002239 | 0.001366 | 0.001174 | 0.001162 | 0.001562 |
| 6 | 0.002221 | 0.001372 | 0.001495 | 0.001571 | 0.001317 |
| 7 | 0.002518 | 0.001236 | 0.00172 | 0.001363 | 0.001248 |

| | | | | | |
|---|---|---|---|---|---|
| 8 | 0.002309 | 0.001303 | 0.001326 | 0.001263 | 0.001624 |
| 9 | 0.002444 | 0.001787 | 0.001132 | 0.001452 | 0.001442 |
| 10 | 0.002782 | 0.002249 | 0.001955 | 0.001297 | 0.001403 |
| 11 | 0.00289 | 0.001439 | 0.001154 | 0.002281 | 0.00158 |
| 12 | 0.002265 | 0.002643 | 0.002723 | 0.0013 | 0.001357 |
| 13 | 0.002409 | 0.001183 | 0.001244 | 0.001373 | 0.001504 |
| 14 | 0.00253 | 0.001522 | 0.001328 | 0.001343 | 0.001421 |
| 15 | 0.002316 | 0.001207 | 0.001122 | 0.00131 | 0.001417 |
| 16 | 0.002381 | 0.001255 | 0.001293 | 0.001312 | 0.001385 |
| 17 | 0.002328 | 0.002469 | 0.001379 | 0.001268 | 0.001391 |

| 16*1024*1024 | | | | | |
|---|---|---|---|---|---|
| Trial | Normal (s) | Degree-4 (s) | Degree-8 (s) | Degree-16 (s) | Degree-64 (s) |
| 1 | 0.036411 | 0.019381 | 0.019855 | 0.019993 | 0.022184 |
| 2 | 0.036135 | 0.021137 | 0.019976 | 0.019375 | 0.021012 |
| 3 | 0.03546 | 0.021706 | 0.019174 | 0.020194 | 0.021236 |
| 4 | 0.03554 | 0.019954 | 0.019862 | 0.019201 | 0.021724 |
| 5 | 0.036554 | 0.021468 | 0.019457 | 0.01912 | 0.021578 |
| 6 | 0.035449 | 0.020018 | 0.019286 | 0.020408 | 0.020836 |
| 7 | 0.037236 | 0.019852 | 0.019061 | 0.019619 | 0.021148 |
| 8 | 0.037002 | 0.021157 | 0.019965 | 0.019338 | 0.02133 |
| 9 | 0.036295 | 0.021874 | 0.021589 | 0.019472 | 0.021935 |
| 10 | 0.035747 | 0.020556 | 0.019689 | 0.019409 | 0.021674 |
| 11 | 0.036831 | 0.020721 | 0.019922 | 0.019528 | 0.022965 |
| 12 | 0.035599 | 0.021265 | 0.019061 | 0.020255 | 0.021266 |
| 13 | 0.036825 | 0.019904 | 0.01946 | 0.019831 | 0.021519 |
| 14 | 0.036564 | 0.020606 | 0.019031 | 0.019747 | 0.020263 |
| 15 | 0.036569 | 0.020313 | 0.019032 | 0.019813 | 0.022466 |
| 16 | 0.03626 | 0.020182 | 0.019274 | 0.019653 | 0.021083 |
| 17 | 0.03624 | 0.019377 | 0.019151 | 0.019391 | 0.023279 |

The outliers (red values) have been excluded from each table.

- **Results**

| | | Unrolling Time | | | |
|---|---|---|---|---|---|
| Input-size | Normal (s) | Degree-4 (s) | Degree-8 (s) | Degree-16 (s) | Degree-64 (s) |
| 1024 | $2.17667*10^{-6}$ | $1.162*10^{-6}$ | $1.09467*10^{-6}$ | $1.06467*10^{-6}$ | $1.152*10^{-6}$ |
| 1024*1024 | 0.002367867 | 0.0014678 | 0.001338867 | 0.001361933 | 0.001434267 |
| 16*1024*1024 | 0.0362688 | 0.020548 | 0.019481667 | 0.0196546 | 0.021597067 |

| | | Unrolling Speedup | | | |
|---|---|---|---|---|---|
| Input-size | Normal | Degree-4 | Degree-8 | Degree-16 | Degree-64 |
| 1024 | 100% | 187.32% | 198.84% | 204.44% | 188.94% |
| 1024*1024 | 100% | 161.32% | 176.85% | 173.86% | 165.09% |
| 16*1024*1024 | 100% | 176.50% | 186.16% | 184.53% | 167.93% |

# ➢ Discussion

- **Execution Time Analysis**

The execution time results for the three input sizes (1024, 1024*1024, and 16*1024*1024) reveal notable trends:

- **Small Input Size (1024):** The unrolling times for different degrees were very close, with Degree-16 achieving the fastest average time of $1.06467*10^{-6}$ seconds, corresponding to a speedup of approximately 204.44% compared to the normal loop. This significant improvement can be attributed to the reduction in loop overhead, which is more pronounced for small datasets, where such overhead represents a larger proportion of total execution time.
- **Medium Input Size (1024*1024):** For medium-sized inputs, Degree-8 achieved the best performance with an average time of 0.001338867 seconds, resulting in a speedup of 176.85% over the normal loop. Here, the benefits of unrolling are balanced against the increased demands on the memory hierarchy. The improvements highlight the effectiveness of unrolling in reducing loop iterations while avoiding excessive resource contention.
- **Large Input Size (16*1024*1024):** For large datasets, Degree-8 achieved the best execution time of 0.019481667 seconds, corresponding to a speedup of 186.16%. The performance differences among unrolling degrees are minor due to memory bandwidth and cache utilization becoming the dominant factors in execution time.

- **Speedup Analysis**

- **Small Input Size (1024):** Loop unrolling demonstrated significant improvement, with Degree-16 achieving the highest speedup (204.44%). This highlights the importance of reducing loop control overhead for lightweight computations, where the benefits of unrolling are maximized.
- **Medium Input Size (1024*1024):** Degree-8 offered the best speedup (176.85%), showing a clear advantage in optimizing computational efficiency while maintaining a balance between reduced loop overhead and memory system demands.
- **Large Input Size (16*1024*1024):** At this scale, Degree-8 provided the highest speedup (186.16%), outperforming other degrees. While improvements were evident, the gains were less pronounced due to resource contention and memory bandwidth limitations.

- **Explanation of Results**

The performance trends observed are influenced by the interplay between reduced loop overhead and increased resource demands:

- **Small Input Sizes:** Loop unrolling significantly reduces overhead, leading to substantial gains, especially at higher degrees like Degree-16.
- **Medium Input Sizes:** Moderate unrolling degrees, such as Degree-8, strike a balance between reducing overhead and avoiding excessive cache pressure.

- **Large Input Sizes:** As dataset size grows, memory and cache constraints dominate execution time, limiting the effectiveness of loop unrolling regardless of the degree.

## ➢ Conclusion

Loop unrolling proves to be a practical optimization technique, particularly for small to medium-sized input datasets. Its effectiveness depends on the degree of unrolling and the characteristics of the workload. From this experiment:

- **Small Input Sizes:** Degree-16 demonstrated the best performance, significantly reducing execution time by minimizing loop control overhead.
- **Medium Input Sizes:** Degree-8 achieved the highest speedup, balancing performance gains with resource constraints.
- **Large Input Sizes:** Degree-8 provided the best results, but the overall performance improvements were limited due to memory and cache bottlenecks.

This study underscores the need to carefully select the unrolling degree based on input size and system architecture. For small and medium datasets, loop unrolling delivers noticeable performance improvements. However, for large datasets, its impact diminishes as other factors, such as memory bandwidth and cache utilization, become more critical.